ASSIGNMENT 2 – CO2

OBJECT ORIENTED PROGRAMMING WITH C++

Poojashri RS

192324266

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

1.1 Problem context

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to fees. A concession function that prints the right answer inside the function leaves the stored fee unchanged. The cause is that the function modified a copy.

1.2 Three implementations

```
#include <iostream>

void concessionByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
   std::cout << "  inside (value)   : " << fee << '\n';
}
void concessionByRef(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}
void concessionByPtr(long long* fee, int percent)
{
   if (fee) *fee -= *fee * percent / 100;   // guard against nullptr
}
long long concessionReturn(long long fee, int percent)   // used in Design Decision
{
   return fee - fee * percent / 100;
}

int main()
{
   long long f1 = 45000, f2 = 45000, f3 = 45000, f4 = 45000;
   concessionByValue(f1, 10);   std::cout << "by value  : " << f1 << '\n';
   concessionByRef(f2, 10);     std::cout << "by ref    : " << f2 << '\n';
   concessionByPtr(&f3, 10);    std::cout << "by pointer: " << f3 << '\n';
   f4 = concessionReturn(f4, 10); std::cout << "by return : " << f4 << '\n';
}
```

Prediction before running

Call	My predicted stored fee	Observed stored fee
By value	40000	45000
By reference	40500	40500
By pointer	40000	40500

Actual output (g++ 13.3.0, run by Claude)	inside (value) : 40500 by value : 45000 by ref : 40500 by pointer: 40500 by return : 40500
Note	The value version computes 40500 inside, but the caller's fee stays 45000. Take your own screenshot and save it in Q1/screenshots/.

1.3 Inquiry

Where each version stores its parameter. In all three versions the parameter is stored in the callee's stack frame, which exists only while the function runs. What differs is what that stored parameter contains:

- **By value:** the parameter holds a copy of the number 45000. It is a separate object from the caller's variable. Changes affect only the copy, which is destroyed when the function returns.
- **By reference:** the parameter is an alias (another name) for the caller's object. Compilers normally implement it as a hidden address, so no copy of the data is made and writes go to the caller's variable.
- **By pointer:** the parameter is a real variable that holds the address of the caller's object. Dereferencing it (*fee) reaches the caller's variable. It can be nullptr, so it should be checked.

Why only two change the caller's fee. Reference and pointer give the function access to the caller's object itself. Call by value gives it only a copy, so the stored fee stays 45000.

When to use a const reference. For parameters that are read but not modified and are costly to copy (a Student object, a std::string, a vector), for example void print(const Student& s). It avoids the copy and the compiler rejects accidental modification. For small types such as int or long long, pass by value, since copying 8 bytes is as cheap as passing an address.

1.4 Design decision

Several concessions may apply to one student. Two options:

Option	In place (long long& fee)	Return new fee (long long f(long long, int))
Chaining	Separate statements: apply(f,10); apply(f,25);	Composable: fee = apply(apply(fee,10),25);
Side effects	Modifies caller's data; harder to test and undo	No side effects; easy to unit-test; original fee stays available
Cost	No copy	Copy of one long long (8 bytes), negligible
Risk	A mistaken call silently changes stored data	Caller can forget to assign the result

1.5 Predict the output

```
#include <iostream>
void change(int a, int& b, int* c)
{
   a += 10;
```

```
    b += 10;
    *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

My predicted output (before running)	1 12 13 1 100 16 13 6
Actual output	1 12 13 1 100 16 13 6
Rules behind the actual output	change(a, b, &c): a is a copy, so main's a stays 1. b is a reference, so b becomes 2+10 = 12. c is changed through its address, so c becomes 3+10 = 13. Output: 1 12 13. bigger(a, c): a = 1 and c = 13, so it returns a reference to c, and the assignment sets c = 100. a is unchanged. Output: 1 100.
Which variables changed in each call	change(): b and c changed, a did not. bigger(a,c) = 100: only c changed (the larger one).
Default arguments used by each call to show()	show(1): y = 5 and z = 10 defaults used, 1+5+10 = 16. show(1, 2): only z = 10 used, 1+2+10 = 13. show(1, 2, 3): no defaults used, 1+2+3 = 6.

1.6 Complete the code

TODO 1 completed :

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;     // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;     // TODO 1 done: 45000 - 4500 = 40500
}
int main()
{
```

```
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Expected output:

```
after call by value : 45000
after call by reference: 40500
```

Actual output	after call by value : 45000 after call by reference: 40500
Result	Matches the expected output exactly.

Why call by value leaves 45000: the parameter fee in applyByValue is a new variable initialised with a copy of hostelFee. The subtraction changes that copy, which is destroyed when the function returns. hostelFee in main was never touched.

1.7 Alternative concept and complexity analysis

Return-the-new-fee version. long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; } is used as hostelFee = applyConcession(hostelFee, 10); The input is copied once, and the result is copied back (usually elided or kept in a register).

Time: O(1) in every version, since only a multiplication, a division and a subtraction are done.
Space copied per call:

Parameter type	By value	By reference	By const reference
long long (8 bytes)	8 bytes copied	no copy (alias; about 8-byte address at most)	no copy; read-only
Large Student object (S bytes, found with sizeof)	S bytes copied every call (O(S))	no copy; can modify caller's object	no copy; cannot modify
Passing risk	None for the caller	Caller's data may change	None for the caller

Measure sizeof(Student) for a sample struct with, for example, char name[64]; long long fee; int marks[100]; and report the number to support the table. Pass-by-value of such an object copies hundreds of bytes per call; by reference or const reference it costs one address.

sizeof measurement (x86-64, g++ 13.3.0)	sizeof(long long) = 8 sizeof(Student) = 472 sizeof(Student*) = 8
Meaning	Passing this Student by value copies 472 bytes on every call, against 8 bytes for the address behind a reference or pointer. A long long costs 8 bytes either way.

Q2 – Return by Reference and Its Dangers

2.1 Problem context

A marks register stores class marks in an array. highest() returns a reference to the top mark so that moderation (+2) can be applied directly. A second function wrongly returns a reference to a local variable.

2.2 Implementation

```
#include <iostream>
#include <vector>

int& highest(int marks[], int n)              // returns alias to the element
{
   int idx = 0;
   for (int i = 1; i < n; ++i)
      if (marks[i] > marks[idx]) idx = i;
   return marks[idx];
}
const int& highestConstRef(const int marks[], int n)   // read-only alias
{
   int idx = 0;
   for (int i = 1; i < n; ++i)
      if (marks[i] > marks[idx]) idx = i;
   return marks[idx];
}
int highestCopy(const int marks[], int n)     // returns a copy
{
   return highestConstRef(marks, n);
}
int& dangling()                               // BUG ON PURPOSE
{
   int local = 99;
   return local;                              // warning: reference to local variable returned
}
int main()
{
   int marks[5] = {56, 78, 91, 67, 85};
   highest(marks, 5) += 2;                    // moderation applied directly
   highest(marks, 5) = highest(marks, 5) + 1; // call on the left of =
   std::cout << "marks[2] = " << marks[2] << '\n';

   int c = highestCopy(marks, 5);             // copy: changing c does not change the array
   c = 0;
   std::cout << "after changing copy, marks[2] = " << marks[2] << '\n';
   // highestConstRef(marks, 5) += 2;         // would not compile: const

   // Changed condition: growing vector
   std::vector<int> v{56, 78, 91};
   int* before = &v[2];
   v.push_back(70);                           // may reallocate
   std::cout << "address changed after push_back? "
             << (before != &v[2] ? "yes" : "no") << '\n';
```

```
    // Do NOT read *before here: it may be a dangling pointer.
}
```

Compile for the warning evidence: g++ -std=c++17 -Wall -Wextra q2.cpp. The expected warning is of the form: warning: reference to local variable ‘local’ returned [-Wreturn-local-addr].

Compiler warning (g++ 13.3.0 -Wall -Wextra)	q2.cpp: In function 'int& dangling()': q2.cpp:25:12: warning: reference to local variable 'local' returned [-Wreturn-local-addr] 25 | return local; | ^~~~~ q2.cpp:24:9: note: declared here 24 | int local = 99; q2.cpp: In function 'int main()': q2.cpp:34:9: warning: variable 'c' set but not used [-Wunused-but-set-variable]
Program output	marks[2] = 94 after changing copy, marks[2] = 94 address changed after push_back? yes
Why moderation worked	The highest mark was 91 (index 2). highest(marks,5) += 2 made it 93, and the second statement (call on the left of =) made it 94. Changing the copy c did not change the array. After push_back the vector moved to new storage, so the old address was no longer valid.

2.3 Explanations required

Returning a reference lets the caller modify the element. highest() returns an alias to marks[idx] and not a copy of the value, so highest(marks,5) += 2 changes the array itself. A function call that returns an lvalue reference can stand on the left of =.

Returning a reference to a local is undefined behaviour. The local variable lives in the function’s stack frame, which is released on return. The reference then names storage that no longer belongs to the variable (a dangling reference). Reading or writing it may give garbage, appear to work, or crash, because the standard places no requirement on the result. Compilers warn with -Wall; treat that warning as an error.

Return type	**int&**	**const int&**	**int (copy)**
What caller gets	Alias to the element	Read-only alias	Independent copy of the value
Can modify the array through the result?	Yes (even on left of =)	No	No (changes a copy only)
Cost	No copy	No copy	Copy (trivial for int, costly for big objects)
Danger	Dangling if the object dies or moves	Same dangling risk	None

2.4 Changed condition: the register is a std::vector that may grow

A std::vector stores its elements in one contiguous block. When push_back needs more room than capacity(), the vector allocates a larger block, moves the elements there and frees the old block. Any reference, pointer or iterator obtained earlier still refers to the old, freed block, so it is invalid and using it is undefined behaviour. Obtaining the reference again after the growth, or keeping an index, avoids the problem.

2.5 Alternative concept and complexity analysis

Instead of returning int&, the function can return the index or a pointer:

```
int  highestIndex(const int marks[], int n);   // calling code:
//   int i = highestIndex(marks, n);  marks[i] += 2;

int* highestPtr(int marks[], int n);          // calling code:
//   int* p = highestPtr(marks, n);   *p += 2;

int& highest(int marks[], int n);             // calling code:
//   highest(marks, n) += 2;
```

Aspect	int& (reference)	int* (pointer)	int (index)
Time to find highest	O(n)	O(n)	O(n)
Extra space	O(1)	O(1)	O(1)
Calling syntax	Cleanest; highest(a,n) += 2	Needs * and null check	Needs marks[i]
After vector grows	Dangling	Dangling	Still valid (index stays the same) if used with the vector again
Empty data	No way to signal “none”	Can return nullptr	Can return -1

All three scan the data once, giving O(n) time and O(1) extra space. The index is the safest choice when the container may grow, because it does not depend on addresses. Reference and pointer are safe only while the storage is not reallocated.

Observation supporting the comparison	In the run above, &v[2] changed after push_back (address changed: yes). A reference or pointer saved before it would therefore have been dangling, while the index 2 would still have identified the same element.

Q3 – Default Arguments and Function Overloading (6 Marks)

3.1 Problem context

A courier counter charges by weight; express delivery and insurance are optional. The developer wrote:

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false, bool insured = false);
```

3.2 The compiler error (ambiguous call)

The declarations themselves are accepted, but a call such as charge(2.5) fails with an error of the form: call of overloaded ‘charge(double)’ is ambiguous. For charge(2.5), the first function matches exactly, and the third also matches exactly, because its two other parameters are filled from defaults. Both are equally good, so the compiler cannot choose. The same happens for charge(2.5, true): the second and third functions are both exact matches. Default arguments make one function behave like several overloads, so the overloads overlap.

Compiler error (g++ 13.3.0)	error: call of overloaded 'charge(double)' is ambiguous note: candidate: 'double charge(double)' note: candidate: 'double charge(double, bool, bool)' error: call of overloaded 'charge(double, bool)' is ambiguous note: candidate: 'double charge(double, bool)' note: candidate: 'double charge(double, bool, bool)'

3.3 Redesign

```
#include <iostream>

struct Parcel {
   double kg;
   bool   express;
   bool   insured;
};

// prototypes (defaults appear ONLY here, once)
double charge(int grams);
double charge(double kg, bool express = false, bool insured = false);
double charge(const Parcel& p);

// (1) weight in grams, whole number
double charge(int grams)
{
   return charge(grams / 1000.0);          // forwards to the kg version
}
// (2) weight in kg, optional extras
double charge(double kg, bool express, bool insured)
{
   double c = kg * 40.0;                   // Rs 40 per kg
   if (express) c += 30.0;
   if (insured) c += 20.0;
   return c;
```

```
}
// (3) a Parcel object
double charge(const Parcel& p)
{
   return charge(p.kg, p.express, p.insured);
}
```

Note that the defaults are written once, in the prototype of the double version, and never repeated in its definition. The three overloads differ in their first parameter type (int, double, const Parcel&), and only one function has defaults, so no call can match two functions equally well.

Six calls to predict before execution:

#	Call	Observed (add a cout line in each function)
1	charge(2500)	int grams, which then forwards to the double kg version. Charge 100
2	charge(2.5)	double kg. Charge 100
3	charge(2.5, true)	double kg. Charge 130
4	charge(2.5, true, true)	double kg. Charge 150
5	charge(2)	int grams (exact match), forwarding to kg: 2 grams = 0.002 kg, charge 0.08
6	charge(2.0f)	double kg (float to double is a promotion, which beats the conversion float to int). Charge 80
7 (extra)	charge(Parcel{2.5, true, false})	Parcel, which forwards to the double kg version. Charge 130

Program output (with a cout tag in each function)	charge(2500) : [int grams] [double kg] 100 charge(2.5) : [double kg] 100 charge(2.5, true) : [double kg] 130 charge(2.5, true, true) : [double kg] 150 charge(2) : [int grams] [double kg] 0.08 charge(2.0f) : [double kg] 80 charge(Parcel) : [Parcel] [double kg] 130

3.4 Inquiry

How the compiler picks among overloaded functions. Candidates with the right name and a viable number of parameters are considered. For each argument, the compiler ranks the conversion needed: (1) an **exact match** (including trivial changes such as adding const or array-to-pointer), (2) a **promotion** (float to double, char or short to int, bool to int), (3) a **standard conversion** (int to double, double to int, int to bool, and so on), (4) a user-defined conversion. The function whose

conversions are at least as good for every argument and better for one wins. If no function is uniquely best, the call is ambiguous. Return types are not considered.

Why defaults must be given from the right. Arguments are matched to parameters by position, from the left. If a middle parameter had a default but a later one did not, there would be no way to omit the middle argument and still supply the later one. So once a parameter has a default, every parameter after it must also have one.

3.5 Alternative concept and complexity analysis

Parcel structure with optional fields (single function):

```
struct Parcel { double kg; bool express = false; bool insured = false; };
double charge(const Parcel& p);                // calls: charge({2.5, true});

// or a function template
template <typename W> double charge(W weight, bool express = false, bool insured = false);
```

Aspect	Overloads with defaults	Single Parcel (or template)
Overload resolution time	Compile time only; zero run-time cost	Compile time only (template instantiation adds compile time)
Space copied per call	Up to 8 + 1 + 1 bytes (padded), or the Parcel by const reference	One const reference (address) or one small struct
Adding a new option (for example fragile)	New parameter or overload; may create new ambiguities; every call site that uses positional arguments is at risk	Add a field with a default value; existing calls keep working
Readability	Short common calls	Named fields (with designated initialisers in C++20) make calls self-documenting

Observation	The redesigned set compiled with no warnings under -Wall -Wextra and every call selected exactly one function, so there was no ambiguity. Note the effect of charge(2): the literal 2 is an int, so it is read as 2 grams and not 2 kg. A caller who means kilograms must write 2.0.

Q4 – Specifying a Class with an Array Inside It

4.1 Problem context

Each SavingsAccount remembers its last 10 transactions for the ATM mini-statement. Older transactions are discarded.

4.2 Class specification (shifting array version)

```
#include <iostream>

class SavingsAccount {
   static const int MAX = 10;     // change to 50 for the Design Decision
   long   accNo;
   double balance;
   double txn[MAX];              // signed: + deposit, - withdrawal
   int    count;                 // number of valid entries (0..MAX)
   void   record(double t);      // private helper
public:
   SavingsAccount(long no, double opening)          // defined inside the class
      : accNo(no), balance(opening), count(0) {}
   double getBalance() const { return balance; }    // defined inside the class
   void   deposit(double amt);                      // defined outside (scope resolution)
   bool   withdraw(double amt);
   void   miniStatement() const;
};

void SavingsAccount::record(double t)
{
   if (count == MAX) {                          // 11th transaction: shift left, drop oldest
      for (int i = 1; i < MAX; ++i) txn[i - 1] = txn[i];
      txn[MAX - 1] = t;
   } else {
      txn[count++] = t;
   }
}
void SavingsAccount::deposit(double amt)
{
   if (amt <= 0) return;
   balance += amt;
   record(amt);
}
bool SavingsAccount::withdraw(double amt)
{
   if (amt <= 0 || amt > balance) return false;
   balance -= amt;
   record(-amt);
   return true;
}
void SavingsAccount::miniStatement() const
{
   std::cout << "Mini-statement for " << accNo << '\n';
   for (int i = 0; i < count; ++i)
      std::cout << "  " << (txn[i] >= 0 ? "CR " : "DR ")
```

```
            << (txn[i] >= 0 ? txn[i] : -txn[i]) << '\n';
   std::cout << "  Balance: " << balance << '\n';
}

int main()
{
   SavingsAccount a(1001, 5000);
   for (int i = 1; i <= 12; ++i) {          // 12 transactions: first two must disappear
      if (i % 3 == 0) a.withdraw(100 * i);
      else          a.deposit(100 * i);
   }
   a.miniStatement();
}
```

Output for 12 transactions (shifting version)	Mini-statement for 1001 DR 300 CR 400 CR 500 DR 600 CR 700 CR 800 DR 900 CR 1000 CR 1100 DR 1200 Balance: 6800
Check	Transactions 1 and 2 (CR 100, CR 200) were discarded and only transactions 3 to 12 are listed, in order. Balance: 5000 + 100 + 200 − 300 + 400 + 500 − 600 + 700 + 800 − 900 + 1000 + 1100 − 1200 = 6800, which matches.

4.3 Inquiry

Why the array must be private. The class has an invariant: count is the number of valid entries, and the entries are in time order with at most 10 kept. If the array were public, any code could write txn[7] = 1e9 or change count and break the statement or the balance relationship. Keeping it private means only deposit() and withdraw() (through record()) can change it, and the layout can be changed later (for example to a circular buffer) without touching callers.

What happens at the 11th transaction.

	Shift the array	**Circular index (start, count)**
Idea	Move elements 1..9 down to 0..8 and put the new one at index 9	Overwrite the oldest slot at (start+count)%MAX and advance start
Cost per add when full	O(k), with k = 9 copies (O(MAX) in general)	O(1)
Printing	Simple loop from 0	Loop i from 0 with index (start+i)%MAX
Extra space	None	One extra int (start)

4.4 Design decision: the last 50 transactions

Make the capacity a single named constant (static const int MAX) and use it in every loop and bound, as the code above does. Then moving from 10 to 50 is a one-line change and the rest of the class is untouched. The shifting design then does up to 49 copies per transaction, which is still small for one account, but the circular design stays O(1) and is the better choice if the capacity or the transaction rate grows. Defend your choice using your own timings from Q7.

4.5 Alternative concept and complexity analysis

Circular buffer version (only the data members and record()/miniStatement() change):

```
   double txn[MAX];
   int    start;   // index of the oldest entry
   int    count;

void SavingsAccount::record(double t)
{
   txn[(start + count) % MAX] = t;
   if (count < MAX) ++count;
   else             start = (start + 1) % MAX;     // overwrite oldest
}
// printing:  for (int i = 0; i < count; ++i) use txn[(start + i) % MAX]
```

std::deque version: keep std::deque<double> txn; on each transaction call push_back(t) and, if size() > 10, pop_front(). Both operations are O(1).

	Shifting array	**Circular buffer**	**std::deque**
Time to add	O(k) when full (k = MAX-1)	O(1)	O(1) (amortised, allocates blocks)
Space	MAX doubles, fixed	MAX doubles + start	About 10 elements plus block bookkeeping; heap allocation
Complexity of code	Simplest	Index arithmetic with %	Simple code, library handles storage
Copy of account object	Trivial copy	Trivial copy	Copies the deque (heap)

Circular buffer version, same 12-transaction test	Output identical to the shifting version (compared with diff: no differences). Balance 6800 and the same 10 entries from DR 300 to DR 1200.
Timing	See the Q7 benchmark table: the circular version stays at about 0.45 to 0.5 ms per 100,000 adds at every capacity, while the shifting version grows with capacity. The std::deque version was not measured.

Q5 – Static Members and Arrays of Objects

5.1 Design from the principle

Principle: data that belongs to the whole class, not to any one object, should exist only once however many objects are created. Applying it to the bus:

- **issued** (how many seats have been given out) belongs to the whole class, so it is a **static data member**. CAPACITY is also class-wide and constant, so it is **static const**.
- **seat** and **name** belong to one ticket, so they stay ordinary (non-static) members.
- **count()** only reports class-wide data, so it is a **static member function** and can be called as BusTicket::count() with no object.
- **Where a static data member is defined:** it is only declared inside the class. It must be defined exactly once outside the class, in one .cpp file, as int BusTicket::issued = 0; Without this definition the linker reports an undefined reference. (A static const int with an in-class constant initialiser, like CAPACITY, can be used as a value without a separate definition unless it is odr-used, for example bound to a reference.)

Why a static member function cannot use this or non-static members. A static member function is not called on an object, so there is no object for this to point to. A non-static member such as seat or name exists once per object, so without an object the function cannot know which one is meant. It can use only static members and its parameters.

How an array of objects is created. BusTicket trip[4] = {BusTicket("Asha"), ...}; creates four objects, and the constructor runs once per element, in index order. Without an initialiser list, the class needs a default constructor, or the array declaration will not compile. Here the constructor is explicit and takes a name, so each element is initialised explicitly.

Count when tickets are created in a loop or a function. The static counter exists once for the whole program, so it keeps increasing across loop iterations and function calls. It does not go down when ticket objects are destroyed (a destructor would have to do that explicitly), so tickets created in a function that returns still count as issued.

5.2 Predict the output

```
#include <iostream>
class Token {
   static int next;
   int id;
public:
   Token() { id = ++next; }
   int get() const { return id; }
   static int issued() { return next - 100; }
};
int Token::next = 100;
int main()
{
   Token counter[3];
   Token extra;
   std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
             << ' ' << Token::issued() << '\n';
}
```

My predicted output (before running)	100 101 102 3
Actual output	101 103 104 4
Why the counter starts at 100; constructor runs	int Token::next = 100; is the single definition of the static member and is initialised before main starts. The constructor ran 4 times (3 array elements plus extra). Each run pre-increments next, so the ids are 101, 102, 103 and 104. counter[0] = 101, counter[2] = 103, extra = 104, and issued() = 104 − 100 = 4.
Differences between my prediction and the actual output	1 2 2 1

5.3 Complete the code

```
#include <iostream>
#include <string>

class BusTicket {
   static int issued;                    // shared by all BusTicket objects
   static const int CAPACITY = 3;
   int seat;
   std::string name;
public:
   explicit BusTicket(const std::string& n) : seat(0), name(n)
   {
      if (issued < CAPACITY)             // TODO 1
         seat = ++issued;                // seat stays 0 (waiting list) otherwise
   }
   static int count()
   {
      return issued;                     // TODO 2
   }
   void print() const
   {
      std::cout << name << ": "
            << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting")) << '\n';
   }
};
int BusTicket::issued = 0;               // definition of the static data member

int main()
{
   BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                        BusTicket("Chitra"), BusTicket("Dev")};
   for (const BusTicket& t : trip) t.print();
```

```
    std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Actual output	Asha: Seat 1 Bala: Seat 2 Chitra: Seat 3 Dev: Waiting Issued: 3
Result	Matches the expected output exactly. Compiled with no warnings under -Wall -Wextra.

Test: tickets created inside a function called twice. Add this to a separate test file (keep the class unchanged):

```
void bookTwo(const std::string& a, const std::string& b)
{
    BusTicket x(a), y(b);
    x.print();
    y.print();
}
int main()
{
    bookTwo("P", "Q");
    std::cout << "Issued after call 1: " << BusTicket::count() << '\n';
    bookTwo("R", "S");
    std::cout << "Issued after call 2: " << BusTicket::count() << '\n';
}
```

Predicted output for the two-call test (before running)	
Actual output of the two-call test	P: Seat 1 Q: Seat 2 Issued after call 1: 2 R: Seat 3 S: Waiting Issued after call 2: 3
How the count behaves	The objects x and y are destroyed when bookTwo returns, but the static counter is not decreased, so the count carries over to the second call: 2 after the first call, and only one more seat (R) was left in the second call, so S went to the waiting list.

Array of 6 tickets (CAPACITY = 3)	A: Seat 1 B: Seat 2 C: Seat 3 D: Waiting E: Waiting F: Waiting Issued: 3

5.4 Alternative concept and complexity analysis

Global counter: int g_issued = 0; used directly in the constructor. **TicketOffice object:** class TicketOffice { int issued = 0; int capacity; public: int nextSeat(); }; and BusTicket(const std::string& n, TicketOffice& office) calls office.nextSeat().

	Static data member	**Global variable**	**TicketOffice object**
Time per ticket	O(1)	O(1)	O(1)
Space	One counter	One counter	One counter per office (plus the reference passed in)
Encapsulation	Private to the class; only BusTicket code changes it	Visible to the whole program; any code can change it	Private to TicketOffice; controlled through its interface
Two buses at once	No: one counter for all BusTicket objects	No	Yes: one TicketOffice per bus, each with its own capacity
Constructor signature	Simple	Simple	Needs the office passed in every time

Conclusion to defend: a static member is the best fit when there is exactly one bus. If the college has several buses, the TicketOffice design is better since the shared data is per bus, not per class.

Observation	Counter value stayed correct across array creation and two function calls with the static design. A global would give the same numbers, but any code could change it; for two buses at once, a TicketOffice per bus would be needed because the static counter is shared by all BusTicket objects.

Q6 – Friend Functions and Inline Functions

6.1 Design of the comparison

- **Why a friend is needed.** minutesBetween(a, b) and earlier(a, b) treat both objects equally and need the private h and m of both. A member function would read the private data of *this* directly but would make one of the two objects the "owner", and it would be written asymmetrically (a.earlier(b)). A friend function is a non-member that the class explicitly grants access to its private members, so it can read both arguments' private data with symmetric call syntax.
- **Friend versus member.** A member is called on an object (a.minutesTo(b)), has a hidden this, and has access to private data by default. A friend is called like an ordinary function (minutesBetween(a, b)), has no this, and can access private members only because the class declared it a friend. A friend is not a member and is not inherited.
- **What inline asks the compiler to do.** inline is a request, not a command, to substitute the function body at the call site instead of generating a call. It removes call overhead (argument passing, jump, return) and lets the optimiser see across the call. The compiler may ignore it (for large or recursive functions or at -O0), and may inline functions that are not marked inline. In modern C++ the keyword's binding effect is that the function may be defined in several translation units (it is needed for definitions in headers). Functions defined inside a class are implicitly inline.
- **What is lost if h and m were public.** Encapsulation. Any code could create invalid times such as 27:95 by assigning h = 27, the class could not enforce that $0 \le h < 24$ and $0 \le m < 60$, and the internal representation (for example, changing to "minutes since midnight") could not be changed without breaking all callers.

6.2 Complete the code

```
#include <iostream>
#include <cstdlib>      // std::abs

class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int  minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
   return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));   // TODO 1
}
bool earlier(const Time& a, const Time& b)
{
   return a.h * 60 + a.m < b.h * 60 + b.m;                 // TODO 2
}

int main()
{
   Time departure(9, 45), arrival(13, 10);
   std::cout << minutesBetween(departure, arrival) << ' '
             << std::boolalpha << earlier(arrival, departure) << '\n';
```

```
}
```

Expected output: 205 false

Actual output	205 false
Result	Matches the expected output. Compiled with no warnings under -Wall -Wextra.

6.3 Correctness check

Calculate each answer by hand first, then run the tests.

Pair (a, b)	Case	Observed
9:45, 13:10	Morning to afternoon	205, true
11:59, 12:01	Either side of noon	2, true
12:01, 11:59	Reversed (abs check)	2, false
12:00, 12:00	Same time twice	0, false

6.4 Growth investigation: timing 10,000,000 calls

```
#include <chrono>
#include <cstdlib>
#include <iostream>

class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int  minutesInline(const Time&, const Time&);
   friend int  minutesNoInline(const Time&, const Time&);
};
inline int minutesInline(const Time& a, const Time& b)
{ return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m)); }

__attribute__((noinline))                  // GCC/Clang: forbids inlining
int minutesNoInline(const Time& a, const Time& b)
{ return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m)); }

template <typename F>
double timeIt(F f, long long& sink)
{
   auto t0 = std::chrono::steady_clock::now();
   for (int i = 0; i < 10000000; ++i) {
      Time a(i % 24, i % 60), b((i / 7) % 24, (i / 3) % 60);   // varying data, no constant folding
      sink += f(a, b);
   }
```

```
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::milli>(t1 - t0).count();
}
int main()
{
    long long s1 = 0, s2 = 0;
    double t1 = timeIt([](const Time& a, const Time& b){ return minutesInline(a, b); },   s1);
    double t2 = timeIt([](const Time& a, const Time& b){ return minutesNoInline(a, b); }, s2);
    std::cout << "inline    : " << t1 << " ms  (checksum " << s1 << ")\n";
    std::cout << "no inline : " << t2 << " ms  (checksum " << s2 << ")\n";
}
```

Compile twice: g++ -std=c++17 -O0 timing.cpp -o t0 and g++ -std=c++17 -O2 timing.cpp -o t2. Run each several times (for example five) and report the median. The checksum is printed so that the compiler cannot remove the loop.

Median of 5 runs (ms)	**Inline version**	**noinline version**	**Ratio noinline ÷ inline**
-O0	172.2	175.4	1.02 (no clear difference)
-O2	46.6	45.7	0.98 (no clear difference)

Measured on x86-64, g++ 13.3.0, 10,000,000 calls per run; five runs each (inline / noinline, ms): -O0: 172.2/171.1, 171.3/170.3, 171.5/175.4, 205.3/199.1, 186.8/206.8; -O2: 47.0/45.7, 46.6/45.6, 47.1/45.7, 45.0/45.5, 46.1/49.2. The checksums of both versions were identical (4446446025).

Explanation of the measured result	At -O0 the compiler does almost no inlining, so both versions cost about the same (about 17 ns per iteration). At -O2 both versions run almost 4 times faster, and no difference between inline and noinline can be measured. Run-to-run noise (several ms) is larger than any difference. The loop body is dominated by the integer remainder and division operations that generate the test times, and the call itself costs only a few cycles, so the call overhead is hidden. This shows that the optimisation level matters far more than the inline keyword, and that inline is only a request.

6.5 Alternative concept and complexity analysis

Member-function version: int Time::minutesTo(const Time& o) const { return std::abs((o.h*60+o.m) - (h*60+m)); } and bool Time::isEarlier(const Time& o) const { return h*60+m < o.h*60+o.m; }. Calls: a.minutesTo(b), a.isEarlier(b). (A member may read o.h because access is per class, not per object.) **Getter version:** public int hour() const; int minute() const; and non-member functions that use them.

	Friend functions	**Member functions**	**Public getters + free functions**
Call syntax	minutesBetween(a,b)	a.minutesTo(b)	minutesBetween(a,b)
Access to private data	Direct, for the listed friends only	Direct, as part of the class	Only through getters

	Friend functions	Member functions	Public getters + free functions
Encapsulation	Slightly widened (friend list)	Best: no extra access granted	Exposes the representation through getters
Time per call	O(1)	O(1)	O(1); same after inlining
Extra space	None	None	None
Symmetry	Treats both objects equally	One object is the receiver	Treats both objects equally

Median of 5 runs (ms)	Friend	Member	Getters + free function
-O0	188.2	172.9	226.0
-O2	48.4	49.2	48.1

Observation	At -O2 the three designs run at the same speed within noise (about 48 to 49 ms), as expected once the small functions are inlined. At -O0 the getter version is about 20 to 30% slower because each getter is a real function call. This supports choosing the design on encapsulation grounds, not speed.

6.6 Deployment decision

Suggested position to defend with your own measurements: replace the friend functions with **const member functions** (minutesTo, isEarlier), or keep the friends if you value the symmetric free-function syntax. After inlining, all three designs run at the same speed, so the choice is made on design grounds. Members need no extra access rights and the friend list cannot grow without being noticed. Friends are justified for operations that are naturally symmetric, such as operator<. Whichever you choose, state the evidence (timings at -O2, encapsulation comparison) that supports it.

Evidence to use in my decision	At -O2 friend 48.4 ms, member 49.2 ms, getters 48.1 ms: no meaningful speed difference. Encapsulation: members add no new access rights; getters expose the representation; friends widen access to a named list. Write your decision and defence here in your own words.

Q7 – Diagnose, Redesign and Validate

Program selected: Q4 – Savings Account Mini-Statement (shifting array of transactions).

A. Diagnose the limitation (2 marks)

In the original design, record() moves every stored transaction one place left whenever the array is full and a new transaction arrives. The cost of one deposit or withdrawal is therefore O(k), where k is the number of stored transactions (MAX-1 copies). The size of the array is a compile-time constant, but this cost grows with it: moving from 10 to 50 transactions makes each operation about five times more expensive, and a capacity of thousands (for a longer statement) would make a frequent-update workload slow. The cause is a design characteristic: the position of the oldest element is fixed at index 0, so removing it forces all other elements to move.

Evidence from the original program	The Q4 output for 12 transactions shows the shift: transactions 1 and 2 were dropped and the rest moved left. Timing of 100,000 adds (-O2, median of 5): N = 10: 0.64 ms; N = 50: 1.19 ms; N = 1000: 10.6 ms; N = 10000: 129.9 ms (at -O0: 2.1, 10.9, 222.6 and 2037 ms). Time per add grows with capacity: about 6 ns at N = 10 and about 1.3 microseconds at N = 10000 (-O2). At -O2 the compiler speeds up the copy loop, so the gap is smaller than at -O0, but it still grows in proportion to N.

B. Design objective (2 marks)

Suggested measurable objective: **make adding a transaction O(1), so that the time per add is independent of capacity, while keeping exactly the same mini-statement output.** Measures: (1) time per add at capacities 10, 50, 1000 and 10000 must stay within about 2× of each other; (2) at capacity 10000 the redesigned version must be at least 10× faster than the original; (3) the output for the 12-transaction test is identical; (4) changing capacity requires changing one constant only; (5) extra memory is no more than one int.

C. Design of the revised solution (4 marks)

Change: replace the shifting array with a **circular buffer**. The array stays private and fixed-size, but two integers record where the oldest entry is and how many are valid. A new transaction is written at (start + count) % MAX. When the buffer is full, the write overwrites the oldest entry and start moves forward by one. Nothing is shifted.

C++ features used: private data members, the modulo operator for wrap-around, a static const int MAX for the capacity, and member functions defined outside the class with the scope resolution operator. No library container is required.

Extra memory: one additional int (start). **Trade-offs:** index arithmetic with % makes printing and code slightly more complex and the % operation has a small cost per access; the stored order in memory is no longer chronological, so a direct walk of the array would show rotated order.

SavingsAccount (redesigned)
- accNo : long - balance : double - txn[MAX] : double (private, fixed array) - start : int (index of oldest entry) - count : int (valid entries, 0..MAX) - MAX : static const int

SavingsAccount (redesigned)
+ SavingsAccount(no, opening) + deposit(amt) : void + withdraw(amt) : bool + miniStatement() const : void + getBalance() const : double - record(t) : void (circular write, O(1))

Pseudocode:

```
record(t):
   txn[(start + count) % MAX] = t
   if count < MAX: count = count + 1
   else:           start = (start + 1) % MAX     // oldest overwritten

miniStatement():
   for i from 0 to count-1:
      print txn[(start + i) % MAX]
```

D. Predict before implementing (1 mark)

Write a testable prediction of the improvement and the measure in which you expect it, before running the new code. For example: “the time per add of the original will grow roughly in proportion to capacity (about 5× from 10 to 50 and about 1000× from 10 to 10000), while the circular version will stay almost constant; at capacity 10000 I expect it to be at least __× faster.”

E. Implement and validate (2 marks)

Benchmark harness, using templates so that the capacity can vary, with the same test for both versions:

```
#include <chrono>
#include <iostream>

template <int N> struct ShiftLog {
   double a[N]; int count = 0;
   void add(double t) {
      if (count == N) { for (int i = 1; i < N; ++i) a[i - 1] = a[i]; a[N - 1] = t; }
      else a[count++] = t;
   }
   double last() const { return a[count - 1]; }
};
template <int N> struct RingLog {
   double a[N]; int start = 0, count = 0;
   void add(double t) {
      a[(start + count) % N] = t;
      if (count < N) ++count; else start = (start + 1) % N;
   }
   double last() const { return a[(start + count - 1) % N]; }
};

template <class Log>
double run(long adds, double& sink)
{
```

```
    static Log log;                          // static: avoids a large stack object
    auto t0 = std::chrono::steady_clock::now();
    for (long i = 0; i < adds; ++i) { log.add(double(i)); sink += log.last(); }
    auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::milli>(t1 - t0).count();
}
template <int N>
void compare(long adds)
{
    double s1 = 0, s2 = 0;
    double ts = run<ShiftLog<N>>(adds, s1);
    double tr = run<RingLog<N>>(adds, s2);
    std::cout << "N=" << N << "  shift " << ts << " ms  ring " << tr
              << " ms  (checksums " << s1 << ", " << s2 << ")\n";
}
int main()
{
    compare<10>(100000);
    compare<50>(100000);
    compare<1000>(100000);
    compare<10000>(100000);
}
```

Also run the same 12-transaction test from Q4 on the circular SavingsAccount and check that the statement matches the original. Compile at -O2 (and, for comparison, -O0) and report medians of several runs.

Capacity N	Original (ms)	Redesigned (ms)	Speed-up	Output identical?	Extra memory
10	0.64	0.45	1.4×	Yes	+1 int
50	1.19	0.47	2.5×	Yes	+1 int
1000	10.6	0.46	22.9×	Yes	+1 int
10000	129.9	0.49	267×	Yes	+1 int

Times are for 100,000 adds, -O2, median of 5 runs. The same test at -O0 (median of 5):

Capacity N	Original (ms)	Redesigned (ms)	Speed-up	Output identical?
10	2.13	0.88	2.4×	Yes
50	10.9	0.84	13.1×	Yes
1000	222.6	0.81	276×	Yes
10000	2037.2	0.84	2,437×	Yes

Output check	The circular SavingsAccount printed exactly the same mini-statement as the shifting version for the 12-transaction test (compared with diff: no differences); balance 6800.
Checksums	Both benchmark versions printed the same checksum (4.99995e+09) at every N, so they did the same work.
Machine	x86-64, g++ 13.3.0, Ubuntu 24.04. Timings vary by machine: re-run on yours for your own numbers and screenshots.

F. Defend the final design (2 marks)

Evaluate against your objective in B, using your table in E:

- **Was the objective achieved?** Compare the measured speed-up with the 10× target and check that time per add stays almost constant as N grows.
- **New costs:** one extra int, modulo arithmetic on every access, slightly more complex printing code, and more places where an off-by-one error can occur.
- **Where the original may still be preferable:** for a capacity of 10 the shifting cost is tiny (in my measurement the redesign was only about 1.4× to 2.4× faster, a difference of a few nanoseconds per add). The shifting version also keeps the data in chronological order in memory, which makes the code easier to read, debug and extend, so for N = 10 simplicity may win.
- **If the improvement is smaller than predicted:** explain why (for example the compiler turned the shifting loop into a fast block copy or vectorised it, or the cache made the copies cheap).

Evidence to use in my evaluation	Objective (1): the redesigned time stayed between 0.45 and 0.49 ms across N = 10 to 10000 at -O2, which is within 2× (achieved). Objective (2): at N = 10000 the speed-up was about 267× at -O2 and 2,437× at -O0 (target 10×, achieved). Objective (3): output identical (achieved). Objective (4): one constant (MAX) controls the capacity (achieved). Objective (5): one extra int (achieved). At N = 10 the gain is small (1.4× at -O2, about 0.2 ms over 100,000 adds, or 2 ns per add), so the simple original loses little at that size. Write your conclusion in your own words.

GITHUB LINK: https://github.com/poojashri07/Object-oriented-c-/tree/main/DSA0103-OOP-CPP-Assignment2